

Arbres

Partie I

1. Introduction



1.1 Un arbre c'est quoi ?

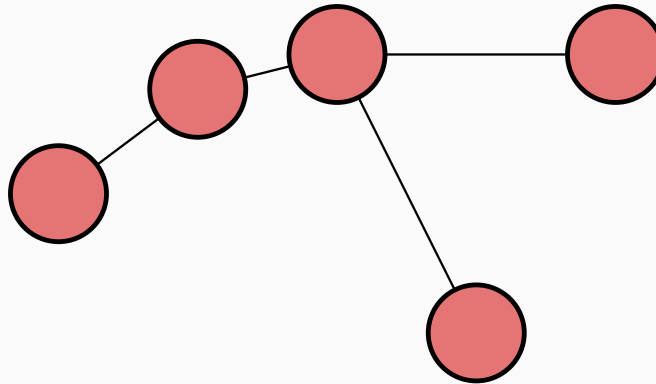
Il y a plusieurs manières de définir un arbre, la plus simple reste la suivante :

Definition 1.1.1 (Arbre)

Un **arbre** est un **graphe** (V, E) qui admet les 2 propriétés suivantes :

- Il est **acyclique**
- Il est **connexe**

Remarquez que le graphe n'est pas nécessairement orienté.



Ceci est un arbre ...

1.2 Propriétés

Soit l'arbre $\mathcal{A} = (V, E)$, alors il vérifie les propriétés suivantes:

- Le nombre d'arêtes ($|E|$) est toujours égal au nombre de noeuds ($|V|$) moins 1. C'est à dire $|E| = |V| - 1$.
- Le chemin entre 2 noeuds est toujours **unique**. (cf. *graphe acyclique*)

Il arrive donc en général qu'un arbre soit défini sur la base de l'une de ces 2 propriétés.

Note

On dit qu'un graphe est une **forêt** lorsque chacune de ses composantes connexes est un arbre.

Corrolaire. Si on enlève une arête d'un arbre, alors l'arbre se décompose en 2 composantes connexes qui sont eux mêmes des arbres, donc le graphe devient une **forêt**.

1.3 Vocabulaire

Definition 1.3.1 (Sous-arbre)

Comme dans les graphes, on dit que $\mathcal{B}$ est un **sous-arbre** de $\mathcal{A}$ si et seulement si tout les noeuds et arêtes de $\mathcal{B}$ sont aussi des noeuds et arêtes de $\mathcal{A}$.

Quelques points de vocabulaire :

Definition 1.3.2 (Feuille)

Une **feuille** d'un arbre est un noeud qui admet un seul voisin. En d'autres termes, si ce noeud s'appelle v , alors $\deg(v) = 1$.

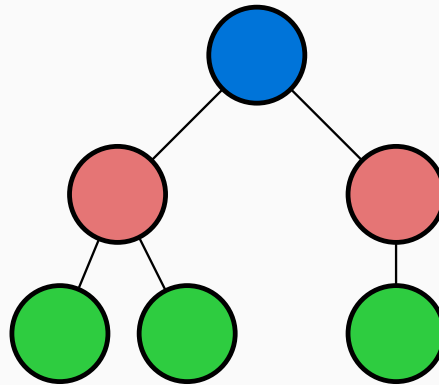
1.4 Arbre enraciné (*rooted tree*)

Definition 1.4.1 (Arbre enraciné)

Un arbre est dit **enraciné** lorsqu'un de ses noeuds est décrit comme étant la **racine**.

Dans un arbre enraciné, on place la racine tout en haut, puis ses voisins en bas, et on fait cela récursivement sur chacun des noeuds jusqu'à arriver aux feuilles.

Exemple.



La racine est en bleu, les feuilles en vert ...

1.5 Terminologie des arbres enracinés

On se place dans le cadre d'un arbre enraciné à partir de maintenant,

Definition 1.5.1 (Parent)

Le **parent** d'un noeud v est le premier noeud après v dans le chemin de v à la racine. Ce parent est **unique**.

La racine n'a pas de parent ...

Definition 1.5.2 (Fils)

Analoguement à la définition d'un parent, supposons que u est le parent de v , alors v est un **fil**s de u .

Un noeud peut avoir plusieurs fils mais a un unique parent ...

Definition 1.5.3 (ancêtres)

Les **ancêtres** d'un noeud u est l'ensemble des noeuds contenu dans le chemin de u vers la racine.

Un noeud est donc considéré comme son propre ancêtre :)

1.6 Profondeur et hauteur

La différence entre **hauteur** et **profondeur** est assez subtile, en effet :

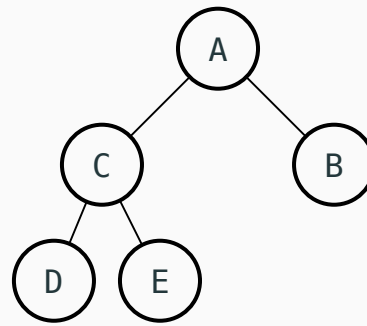
Definition 1.6.1 (Profondeur)

La **profondeur** d'un noeud v est le nombre d'arêtes dans le chemin entre v et la racine de l'arbre.

En particulier la racine est de profondeur 0 ...

Definition 1.6.2 (Hauteur)

La **hauteur** d'un noeud v est le nombre d'arêtes dans le chemin entre v , et la feuille la plus basse dans le sous-arbre obtenu à partir de v .



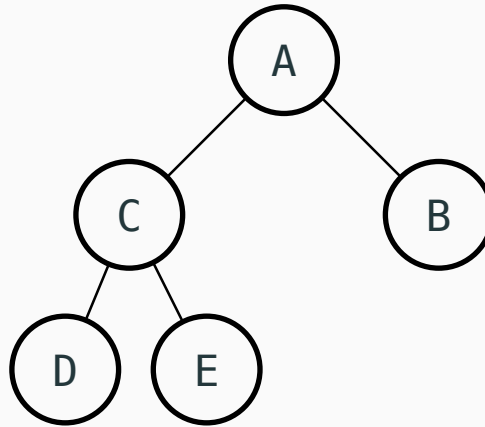
B est de profondeur 1 et de hauteur 0.

2. En pratique

2.1 Représentation

De manière classique, un arbre étant un graphe, on peut le représenter sous la forme d'une liste d'adjacence.

On peut chercher à aller plus loin pour faciliter le parcours en *DFS*, en enracinant l'arbre, et en gardant simplement les enfants de chacun des noeuds.



Liste d'adjacence simplifiée : $\{A:[C, B], B:[], C:[D, E], D:[], E:[]\}$

2.2 Représentation

Il est alors plus facile d'écrire l'algorithme de *DFS* :

```
type tree_t = list[list[int]] # Structure de l'arbre
def dfs(tree : tree_t, root : int):
    """
    Parcours DFS du sous-arbre engendré par `root`
    """
    # On peut faire quelque chose à chaque noeud si on le souhaite...
    for node in tree[root]:
        dfs(tree, node)
```

3. Crédits

Auteurs et images : Bilel MAHDJOUR

Pour le **Bureau des technologies** de **Polytech Nice Sophia**

4. Sources et ressources

- [CPH book - trees](#)
- [USACO guide - Introduction to tree algorithms](#)